C++ BASIC DATA TYPES

The Beginner’s Friendly Mastery Guide

CodeHive Academy

Standard Documentation v1.02

1. Why Data Types Matter?

In the world of C++, data isn't just "data." Everything must be categorized. This is because C++ is a **Strongly Typed Language**. Before a program can run, the computer needs to know exactly how much memory to reserve for every piece of information.

The Computer's Perspective:

- If you say "25," is that a number to calculate or a text character?
- If you say "3.14," do you need high precision for science or just a rough estimate?

By defining a **Type**, you provide the blueprint. In this guide, we will explore how CodeHive organizes these categories into Numbers, Booleans, Characters, and Strings.

2. Working with Numbers (Numerical Types)

Numbers are the backbone of almost every algorithm. C++ separates numbers into two main families: **Integers** (Whole numbers) and **Floats** (Decimals).

Comparison of Basic Numerical Types

Type	Description	Memory	Usage at CodeHive
`int`	Whole numbers	4 Bytes	Counting employees, years.
`float`	Decimal (Simple)	4 Bytes	Basic height, weights.
`double`	Decimal (High)	8 Bytes	Currency, precision math.
`short`	Small Range	2 Bytes	Optimizing small counts.
`long`	Huge Range	8 Bytes	World populations.

CodeHive Practice: Most developers use `int` for counts and `double` for decimals. Only use `float` if you are working on a system with very limited memory.

3. Numbers in Action

Let's see how these numeric types look in a real CodeHive administrative script.

```
#include <iostream> using namespace std; int main() { int employees = 50;
float officeArea = 1234.5; double revenue = 98765.4321; cout << "CodeHive
has " << employees << " employees." << endl; cout << "The office area is "
<< officeArea << " sq.m." << endl; cout << "Annual revenue: $" << revenue
<< endl; return 0; }
```

Check: Notice how `double` preserves much more detail after the decimal point than `float` does during high-precision math.

4. The Logic of Booleans (bool)

Named after George Boole, this data type represents the simplest form of choice: **True** or **False**. In C++, these are the gatekeepers of decision-making logic.

```
bool isHiring = true; bool isRemote = false; cout << "Is hiring? " <<
isHiring; // Prints 1
```

C++ Logic Internal:

1 → `true`

0 → `false`

Internally, C++ treats any non-zero number as true, but by default, it uses 1.

At CodeHive, we use these for checking states, such as `isLoggedIn`, `isPaid`, or `isCourseComplete`.

5. Characters (char)

The `char` data type stores exactly **one** character. This could be a letter, a number, or a symbol.

> **Rule:** Characters must be enclosed in **Single Quotes** (`' '`). Double quotes are reserved for strings.

```
char grade = 'A'; char symbol = '#'; char digit = '7'; // This is a character '7', not the number 7!
```

Fun Fact: ASCII

Computers don't actually know what 'A' is. They store 'A' as the number **65**. This numerical map is called ASCII. You can even perform math on characters!

6. Textual Strings (string)

Unlike basic types, `string` is an **Object**. It is a collection of characters used to store names, sentences, or paragraphs.

Required Setup

To use strings professionally, CodeHive developers include the string library:

`#include <string>`

```
string company = "CodeHive"; string motto = "Code, Learn, Repeat"; cout << company << " says: " << motto;
```

Memory Tip: Strings can grow and shrink dynamically as you add or remove text. This makes them much more flexible than a simple `char` array.

7. Capturing User Text

When asking users for their names or feedback, standard `cin` has a major limitation: it stops at the first space.

```
string name; cout << "Enter your full name: "; getline(cin, name); // Captures the entire line including spaces
```

CodeHive Safety: Buffer Clearing

If you use `cin >>` before `getline()`, a "newline character" often stays in the keyboard buffer, causing `getline` to skip. Use `cin.ignore()` to clear it.

```
int years; cin >> years; cin.ignore(); // Clear whitespace getline(cin, fullName);
```

8. The 'auto' Keyword (Modern C++)

Introduced in C++11, `auto` tells the compiler to **infer** the type based on the value you provide. It's like the compiler doing the thinking for you.

```
auto employees = 50; // Compiler sees 50, sets to int auto revenue =
1234.56; // Compiler sees decimal, sets to double auto company =
"CodeHive"; // Sets to string (technically const char*)
```

CodeHive Advice: For beginners, it's better to type the actual type (`int`, `string`) so you learn how memory works. Use `auto` later for complex iterator types!

9. Case Study: CodeHive Dashboard

Let's combine all types into a single cohesive program representing a company overview.

```
#include <iostream> #include <string> using namespace std; int main() { string companyName = "CodeHive"; int employeeCount = 50; float avgRating = 4.8; bool isActive = true; char regionCode = 'G'; // Displaying Dashboard cout << "--- " << companyName << " STATS ---" << endl; cout << "Employees: " << employeeCount << endl; cout << "Avg Rating: " << avgRating << "/5" << endl; cout << "Active: " << (isActive ? "YES" : "NO") << endl; return 0; }
```

10. Visualizing the RAM

When you run the CodeHive Dashboard code above, here is how the computer's memory (RAM) is organized:

Variable Name	Type	Binary Space (approx)
`companyName`	string	32+ Bytes (Heaps)
`employeeCount`	int	4 Bytes (Stack)
`avgRating`	float	4 Bytes (Stack)
`isActive`	bool	1 Byte (Stack)
`regionCode`	char	1 Byte (Stack)

This layout efficiency is why C++ is used for high-performance software like Chrome, Photoshop, and AAA Games.

11. CodeHive Laboratory: Exercises

It’s time to get your fingers on the keyboard. Solve these three levels of challenges to test your understanding.

Level 1: The Identity Card

Create variables for your Name (string), Age (int), and Gender (char 'M' or 'F'). Display them in a clean ID format.

Level 2: The Logic Switch

Assign a boolean variable `isStudent` to true. Ask the user if they are a student. If 1, greet them as a CodeHive student.

Level 3: Precise Revenue

Create a `double` for daily revenue. Ask for inputs over 3 days, calculate the total, and print it using `auto` for the result variable.

12. Advanced usage of auto

While we initially learned `auto` is a shortcut, it is technically a **Place-holder Type Specifier**. In modern CodeHive workflows, we use it for readability when the type name is very long.

```
// Hard to read: std::vector<std::string>::iterator it = list.begin(); // Clean CodeHive style: auto it = list.begin();
```

The compiler replaces `auto` during compilation, so there is **zero** performance penalty. Your program runs just as fast!

13. Summary Checklist

Before moving to the next module, ensure you can check off every item on this CodeHive competency list:

- ✅ I know the difference between `int` and `double`.
- ✅ I understand that `float` is for basic decimals.
- ✅ I can capture `getline()` text with spaces.
- ✅ I remember single quotes for `char` and double for `string`.
- ✅ I understand that `bool` stores 1 and 0.
- ✅ I know when to use `auto` to save time.

Data types are the grammar of C++. Now that you know the nouns (data types), you are ready to learn the verbs (operators and functions)!

CODE. LEARN. REPEAT.

Mastery of Data Types: Complete

Developed by the CodeHive Curriculum Team